

ROSALIND BIOINFORMATICS

Counting DNA Nucleotides

Problem ID: DNA | Topic: String Algorithms

Report

1. Problem Overview

The Counting DNA Nucleotides problem is an introductory exercise in the Rosalind Bioinformatics Stronghold. It introduces a basic operation in computational biology: determining the nucleotide composition of a DNA sequence.

DNA sequences are composed of four standard nucleotide bases:

- A — Adenine
- C — Cytosine
- G — Guanine
- T — Thymine

Given a DNA string *s* of length at most 1000 nucleotides (nt), the task is to count the number of occurrences of A, C, G, and T, and return the four counts in that order.

Sample Dataset

```
AGCTTTTCATTCTGACTGCAACGGGCAATATGTCTCTGTGTGGATTAAAAAAGAGTGTCTGATAGCAGC
```

Sample Output

```
20 12 17 21
```

2. Approach

The solution uses a dictionary-based counting approach.

The steps are:

1. Read the DNA sequence from the input file.
2. Remove leading/trailing whitespace from the sequence.
3. Initialize a dictionary with the four nucleotide symbols as keys, each with an initial count of zero.
4. Traverse the DNA sequence one nucleotide at a time.
5. Increment the corresponding dictionary value for each nucleotide.
6. Print the counts in the order required by Rosalind: A, C, G, T.

The dictionary provides a simple way to associate each nucleotide with its corresponding count.

3. Algorithm

Given a DNA sequence `dna_string`:

```
Initialize:
    A = 0
    C = 0
    G = 0
    T = 0

For each nucleotide in the DNA sequence:
    increment the count corresponding to that nucleotide

Return the four counts in the order:
    A, C, G, T
```

In the Python implementation, the four counters are stored in a dictionary:

```
counts = {'A': 0, 'C': 0, 'G': 0, 'T': 0}
```

The sequence is then traversed once:

```
for nucleotide in dna_string:
    counts[nucleotide] += 1
```

4. Complexity Analysis

Let n be the length of the DNA sequence.

Time Complexity: $O(n)$

Each nucleotide is visited exactly once.

Auxiliary Space Complexity: $O(1)$

The nucleotide-count dictionary always contains exactly four entries, regardless of the length of the DNA sequence.

The input sequence itself requires $O(n)$ memory because it is read into memory before processing.

Therefore:

- Auxiliary space: $O(1)$
- Input storage: $O(n)$

5. Python Implementation

Below is the finalized implementation, with comments explaining the purpose of each step:

```

"""
Rosalind Problem: Counting DNA Nucleotides (DNA)
-----
Task:
    Given a DNA string s of length at most 1000 nt, count how many
    times each of the four nucleotide symbols ('A', 'C', 'G', 'T')
    occurs in s.

Input:
    A plain text file named 'rosalind_dna(3).txt' containing a single
    line with the DNA string.

Output:
    Four space-separated integers giving the counts of 'A', 'C', 'G',
    and 'T', respectively, in that exact order.
"""

def count_nucleotides(dna_string):
    """
    Count the occurrences of each nucleotide (A, C, G, T).

    Parameters
    -----
    dna_string : str
        A DNA sequence containing A, C, G, and T.

    Returns
    -----
    dict
        Dictionary mapping each nucleotide to its count.
    """
    # Initialize a counter for each of the four nucleotides.
    counts = {'A': 0, 'C': 0, 'G': 0, 'T': 0}

    # Traverse the DNA sequence one nucleotide at a time.
    for nucleotide in dna_string:
        counts[nucleotide] += 1

    return counts

# Read the DNA sequence from the input file.
# .strip() removes leading/trailing whitespace, including newlines.
with open('rosalind_dna.txt') as f:
    dna_sequence = f.read().strip()

# Count the nucleotides.
counts = count_nucleotides(dna_sequence)

# Print the counts in the order required by Rosalind: A, C, G, T.
print(counts['A'], counts['C'], counts['G'], counts['T'])

```

6. Result

The solution was tested using the personalized Rosalind dataset.

The program produce

238 208 234 223

Therefore, the input sequence contained:

Nucleotide	Count
A	238
C	208
G	234
T	223

7. Skills Demonstrated

This introductory problem provided practice in several fundamental programming and bioinformatics concepts.

Python Programming

- Functions
- Dictionaries
- for loops
- String processing
- File input/output

Algorithmic Thinking

- Designing a single-pass counting algorithm
- Selecting an appropriate data structure
- Understanding time and space complexity

Bioinformatics

- Representing DNA sequences computationally
- Processing nucleotide sequences
- Calculating basic nucleotide composition

Scientific Programming Practice

- Writing reusable functions
- Documenting code

- Separating the computational logic from input/output
- Recording and interpreting computational results

8. Conclusion

The Counting DNA Nucleotides problem was solved successfully using a linear-time, dictionary-based counting approach implemented in Python. The solution has:

- $O(n)$ time complexity,
- $O(1)$ auxiliary space complexity,
- a reusable counting function,
- file-based sequence input, and
- output formatted according to the Rosalind requirements.

The solution was successfully submitted to Rosalind and accepted .